BOOTCAMP II · DESAFIO CUMULATIVO

Planejamento Lógico TaskTracker

Entrega Inicial (Fase 1) — Atividade individual

Prazo de entrega no AVA: 13/09/2026, 23h55. Este documento é a fundação da Fase 2 (Git, GitHub e Python) e da Fase 3 (Docker).

1. Identificação e Entendimento do Problema

1.1 Identificação do estudante

Campo	Informação
Nome completo	Felipe Marques
Curso	[preencher]
Campus / turma	[preencher]
E-mail institucional	[preencher]
Disciplina	Bootcamp II
Entrega	Fase 1 — Planejamento Lógico do TaskTracker

Atenção: substitua os campos entre colchetes antes de exportar o PDF. No Google Docs, o arquivo deve se chamar exatamente: Planejamento Lógico TaskTracker - Felipe Marques.

1.2 O problema a ser resolvido

Organizar o dia a dia sem um sistema é mais difícil do que parece. A pessoa (ou um pequeno grupo de estudo) costuma espalhar compromissos em caderno, conversas de WhatsApp, lembretes no celular e anotações soltas. O resultado é sempre o mesmo: tarefa importante some, prazo passa sem aviso, e não dá para saber com clareza o que ainda está pendente, o que é urgente e o que já foi concluído.

Na prática, isso gera três dores concretas:

1. Perda de prazo. Sem um lugar único para guardar título, prioridade e data-limite, o estudante só percebe o atraso quando o trabalho já deveria ter sido entregue.
2. Falta de prioridade. Tudo parece urgente. Sem classificar o que é alta, média ou baixa prioridade, o tempo é gasto no que aparece primeiro, e não no que mais importa.
3. Falta de visibilidade. Não existe um painel simples: quantas tarefas ainda faltam, quantas já foram feitas, quais vencem hoje. Sem esse retrato, a pessoa trabalha no escuro.

O TaskTracker existe para resolver exatamente esse cenário: concentrar o controle das tarefas em um único sistema, com regras claras, para que o usuário nunca dependa só da memória.

1.3 A solução proposta — TaskTracker

O TaskTracker é um sistema de gerenciamento de tarefas simples, pensado primeiro no planejamento lógico e, nas fases seguintes, implementado em Python, versionado no GitHub e, depois, empacotado em Docker.

O sistema é apresentado como já pronto para o usuário: ele abre o TaskTracker, cadastra o que precisa fazer e acompanha a lista na tela. A implementação em Python fica para a Fase 2; nesta etapa o importante é o caminho que a pessoa percorre.

O que o TaskTracker fará:

- Cadastrar tarefas com título, descrição, prioridade (Alta, Média ou Baixa) e prazo.
- Listar tarefas de forma organizada, mostrando status (Pendente ou Concluída).
- Marcar tarefa como concluída, atualizando o status sem apagar o histórico.
- Filtrar a visualização (todas, só pendentes, só concluídas, ou por prioridade Alta).
- Exibir um resumo com totais: cadastradas, pendentes, concluídas e, se houver, atrasadas.
- Impedir cadastros inválidos (título vazio, prioridade fora da lista, prazo em formato inválido).

Em uma frase: você abre o TaskTracker, registra o que precisa fazer, acompanha o que está pendente e marca o que já terminou — tudo com regras que evitam dado errado.

2. Decomposição de Dados e Regras

2.1 Entradas de dados

São as informações que o usuário informa ao sistema (ou que o sistema gera automaticamente no momento do cadastro).

Campo	Tipo	Origem	Obrigatório	Exemplo
Identificador (id)	Número inteiro sequencial	Gerado pelo sistema	Sim (automático)	1, 2, 3
Título	Texto	Usuário	Sim	Entregar relatório de Bootcamp II
Descrição	Texto	Usuário	Não	Montar o PDF da Fase 1 e gravar o vídeo
Prioridade	Texto (lista fechada)	Usuário	Sim	Alta / Média / Baixa
Prazo	Data (AAAA-MM-DD)	Usuário	Sim	2026-09-13
Status	Texto (lista fechada)	Sistema, com atualização pelo usuário	Sim (inicia como Pendente)	Pendente / Concluída
Data de criação	Data e hora	Gerada pelo sistema	Sim (automático)	2026-08-30 15:40
Ação na tela	Escolha do usuário	Usuário	Sim, a cada interação	Nova tarefa, Ver tarefas, Concluir

Observação: o id evita ambiguidade. Se existirem duas tarefas com títulos parecidos, o usuário escolhe pela numeração, não pelo nome.

2.2 Saídas de dados

São as informações que o sistema devolve na tela.

Saída	Quando aparece	Conteúdo
Tela inicial	Ao abrir o sistema e após cada operação	Atalhos para nova tarefa, ver lista, resumo e sair
Cadastro realizado	Após um cadastro válido	Confirmação com o id gerado e o título salvo
Erro de validação	Quando alguma regra é quebrada	Texto objetivo, por exemplo: o título não pode ficar em branco
Lista de tarefas	Ao escolher listar	Cada tarefa em uma linha: id, título, prioridade, prazo, status

Lista vazia	Se ainda não houver tarefas	Não há tarefas cadastradas.
Confirmação de conclusão	Ao marcar uma tarefa pendente	Tarefa 3 marcada como concluída.
Resumo estatístico	Ao pedir o relatório	Totais de cadastradas, pendentes, concluídas e atrasadas
Encerramento	Ao sair	TaskTracker encerrado. Até a próxima.

Exemplo de listagem (saída esperada):

```
ID  Título                          Prioridade  Prazo       Status
1   Entregar relatório de Bootcamp II  Alta        2026-09-13  Pendente
2   Estudar Git e GitHub               Média       2026-09-20  Pendente
3   Revisar algoritmo do cadastro      Baixa       2026-09-10  Concluída
```

2.3 Regras do sistema

O sistema segue estas regras, nesta ordem, sempre que o usuário cadastrar ou visualizar uma tarefa:

4. O título é obrigatório. Se estiver vazio ou só com espaços, o sistema não salva e pede para preencher.
5. A prioridade só pode ser Alta, Média ou Baixa. Qualquer outro valor é recusado.
6. O prazo precisa ser uma data válida. Se o formato não for reconhecido, o sistema pede para corrigir.
7. Toda tarefa nova nasce como Pendente. O usuário não escolhe o status no cadastro.
8. O sistema gera o número da tarefa automaticamente (1, 2, 3…). O usuário não altera esse número.
9. Só é possível concluir uma tarefa que existe e ainda está Pendente. Se o item não existir ou já estiver concluído, o sistema avisa e não muda a lista.
10. As tarefas ficam disponíveis enquanto o programa estiver aberto. Ao fechar, a lista recomeça vazia nesta fase.
11. Tarefa atrasada é a que está Pendente com prazo anterior a hoje. Ela entra no resumo, mas não é apagada.

3. Receita Lógica do Sistema (Algoritmo em Português)

A receita abaixo explica o TaskTracker como se o sistema já estivesse pronto. A linguagem é a de quem vai usar no dia a dia: um passo depois do outro, sem falar de código nem de terminal.

São dois caminhos principais: cadastrar uma tarefa e visualizar as tarefas.

3.1 Passo a passo do cadastro da tarefa

Você abriu o TaskTracker e quer registrar algo que precisa fazer.

12. Na tela inicial, escolha Nova tarefa.
13. Digite o título — um nome curto do que precisa ser feito. Esse campo é obrigatório.
14. Se o título estiver em branco, o sistema avisa que não é possível salvar e pede para você preencher antes de continuar.
15. Se quiser, escreva uma descrição com mais detalhes. Se não quiser, pode deixar em branco.
16. Escolha a prioridade: Alta, Média ou Baixa.
17. Se a prioridade não for uma dessas três opções, o sistema avisa que o valor é inválido e não salva a tarefa.
18. Informe o prazo, ou seja, até quando a tarefa precisa estar pronta.
19. Se a data não for reconhecida, o sistema avisa e pede para você corrigir.
20. Confirme o cadastro.

21. O sistema grava a tarefa, coloca o status como Pendente e mostra uma mensagem de sucesso, com o número da tarefa e o título que você cadastrou.
22. Você volta para a tela inicial e pode cadastrar outra tarefa ou ir ver a lista.

Exemplo: você cadastra "Gravar pitch de 5 minutos", prioridade Alta e prazo 12/09/2026. O TaskTracker confirma: tarefa 1 salva como Pendente.

3.2 Passo a passo da visualização

Agora você quer ver o que já está registrado.

23. Na tela inicial, escolha Ver tarefas.
24. O sistema abre a lista.
25. Se você ainda não cadastrou nada, aparece o aviso: não há tarefas cadastradas.
26. Se houver tarefas, elas aparecem uma a uma, com título, prioridade, prazo e status (Pendente ou Concluída).
27. No topo da lista, um resumo mostra quantas tarefas existem, quantas estão pendentes, quantas já foram concluídas e quantas estão atrasadas.
28. Se quiser, filtre o que aparece na tela: todas, só pendentes, só concluídas ou só prioridade Alta.
29. Se o filtro não encontrar nenhuma tarefa, o sistema avisa que não há resultado para essa escolha.
30. Para marcar uma tarefa como feita, selecione uma tarefa Pendente e confirme a conclusão.
31. O sistema muda o status para Concluída e mostra uma confirmação. A tarefa continua na lista, agora como concluída.
32. Se a tarefa já estava concluída, o sistema avisa e não altera nada.

Exemplo do que aparece na lista:

```
ID  Título                          Prioridade  Prazo       Status
1   Entregar relatório de Bootcamp II  Alta        2026-09-13  Pendente
2   Estudar Git e GitHub               Média       2026-09-20  Pendente
3   Revisar algoritmo do cadastro      Baixa       2026-09-10  Concluída
```

4. Mapeamento Arquitetural para Git e GitHub (Preparação para a Fase 2)

4.1 Desenho da árvore do repositório

A estrutura abaixo é o mapa da pasta que será versionada no GitHub na Fase 2. Cada arquivo tem um papel único, para o repositório nascer organizado.

```
tasktracker-projeto/
├── README.md
├── .gitignore
├── docs/
│   └── planejamento_logico.pdf
└── src/
    └── main.py
```

4.2 Função de cada item

Item	Para que serve
tasktracker-projeto/	Pasta raiz do repositório. É o projeto inteiro no GitHub.
README.md	Primeira tela de quem abre o repositório: nome do sistema, o que ele faz, como executar o programa e um resumo do planejamento.
.gitignore	Impede que o Git versione arquivos temporários, por

	exemplo __pycache__/, .venv/ e arquivos do sistema operacional.
docs/	Pasta de documentação. Separa texto de planejamento do código.
docs/planejamento_logico.pdf	Este documento da Fase 1, exportado em PDF, para ficar registrado junto ao código.
src/	Pasta do código-fonte. Tudo o que for Python fica aqui.
src/main.py	Programa principal: cadastro e visualização das tarefas.

Essa árvore é propositalmente enxuta. Na Fase 1 ainda não existe código; o desenho serve para, na Fase 2, criar as pastas e os arquivos certos já no primeiro commit, em vez de soltar arquivos soltos na raiz.

4.3 Ordem prevista de versionamento (Fase 2)

33. Criar o repositório no GitHub com o nome do projeto.
34. Clonar (ou iniciar o Git na pasta local).
35. Criar as pastas docs/ e src/.
36. Adicionar README.md, .gitignore e o PDF deste planejamento.
37. Fazer o primeiro commit de estrutura (docs + README + .gitignore).
38. Implementar o algoritmo em src/main.py.
39. Fazer commits pequenos e com mensagem clara, por funcionalidade (cadastro e visualização).

4.4 Plano de aprendizado para Git e GitHub

Na Fase 2, o objetivo não é só “subir o código”, e sim aprender a trabalhar com histórico. Espero entender, na prática, o que é um repositório, como o git add escolhe o que entra no próximo pacote de mudanças e como o git commit grava um ponto estável do projeto. Quero usar mensagens de commit que expliquem o porquê da alteração, e não só o nome do arquivo. Também pretendo aprender a criar o repositório no GitHub, enviar (push) as versões e acompanhar o histórico (log) para não perder o rastro do que já foi feito. Se a disciplina avançar para ramificações, quero compreender o papel de uma branch para isolar uma funcionalidade sem bagunçar a versão principal, e o que fazer quando duas alterações mexem no mesmo trecho (conflito). Com isso, o TaskTracker deixa de ser só um arquivo na minha máquina e passa a ter um histórico profissional, que é exatamente o que um desenvolvedor usa no dia a dia.

5. Link do vídeo de apresentação

Status: a gravar. Duração: 5 minutos. Formato: horizontal 16:9, rosto visível, áudio claro. Publicação: YouTube Público ou Não listado (não usar Privado; não usar Shorts).

Link do YouTube:

[colar aqui o link ativo depois da gravação e testar o acesso em aba anônima]

5.1 Roteiro sugerido (5 minutos)

Tempo	Bloco	O que falar / mostrar
0:00 – 0:40	Apresentação	Nome, curso, campus e motivação para o Bootcamp II.
0:40 – 2:00	O problema	Abrir este documento no Google Docs. Explicar a dor: prazos perdidos, falta de prioridade e ausência de um lugar único para as tarefas.

2:00 – 4:00	A solução (receita lógica)	Mostrar entradas, saídas e regras. Percorrer o passo a passo do cadastro e da visualização, como se estivesse ensinando a receita.
4:00 – 5:00	O futuro (Git/GitHub)	Mostrar a árvore de pastas e explicar por que docs/ e src/ ficam separados. Encerrar dizendo que este documento é a base da implementação em Python.

Antes de enviar: abrir o link em outra conta ou em janela anônima e confirmar que o vídeo carrega.

Checklist da entrega no AVA

- Documento no Google Docs com as 5 seções preenchidas
- Identificação (nome, curso, campus, e-mail) completa
- Algoritmo descrito como passo a passo (não como código)
- Árvore do repositório desenhada e explicada
- Vídeo gravado na horizontal, com rosto e áudio nítido
- Link do YouTube colado na seção 5 e testado
- Arquivo exportado em PDF e enviado no AVA até 13/09/2026, 23h55

Este documento é a fundação da Fase 2 (Git, GitHub e Python) e da Fase 3 (Docker). Qualquer mudança de regra daqui para frente deve ser atualizada aqui primeiro, para o código não divergir do planejamento.